Week 09: Programming and Research

Programming task

Session 2: deepen this week’s Python.
Predict → change → repair.
Session 3: apply it to the project.

Research task

No new research assignment.
Use the Week 8 proposal and feedback.
Prepare the handoff as project work.

Open learning_materials/Week_09.ipynb. Follow Start Here; keep your work in that notebook. Email: submit.progmis@gmail.com.

Programming task

Session 2 · Python practice before project application.

1. Predict returned values

Do this

Open #session-2: Predict → Run.
Trace at_least(4/5/6, 5).
Predict the pending list.

Observe and explain

Expect False / True / True.
Identify parameters and return.
Explain the filter condition.

2. Change the filter input

Do this

Change only A’s finished flag to True.
Predict the new pending list.
Compare if finished / if not finished.

Observe and explain

Expected pending list: ["C"].
Explain which record changed group.
Try a new minimum with at_least.

3. Repair and write assertions

Do this

Run buggy_at_least at 4, 5 and 6.
Copy it as fixed_at_least.
Repair the boundary.

Observe and explain

Assert below / equal / above.
Keep an already-correct case.
Explain which assertion catches the bug.

Programming practice: done checklist

Keep in your notebook

Prediction before running
Your changed or repaired code
Observed output or error
One explanation of why it happened

Next step

Optional exploration can wait.
Save the notebook and a backup.
Continue to the project application.
Submit to submit.progmis@gmail.com.

Outside class: complete your change and save the verification evidence.

Project application: your approved improvement

Step	What to do / check
Open Session 3	#project-task: use the Week 8 approved scope; write two expected results before coding.
Implement a small change	Use new cells or a module. Keep an old behavior; check a boundary/failure and read-only snapshot if relevant.
Recheck baseline	Six request outcomes, reports/batch counts, failure snapshots, event order and duplicate refusal.
Peer checks your feature	Provide start/run steps and two cases. Peer tries another input; record finding, response and retest.

Save your actual tests/command, changed files and handoff draft. Practice tests do not test your feature automatically.